

编译原理研讨课实验PR001实验报告

任务说明

成员组成

实验设计

设计思路

实验实现

总结

实验结果总结

编译原理研讨课实验PR001实验报告

任务说明

基于 ANTLR4 实现教学用 C-like 语言 **CACT** 的前端：完成词法与语法分析；保证在发生词法/语法错误时及时报错并以非 0 状态退出；支持基本类型（int/bool/float/double/void）、函数定义/调用、数组、条件与循环等语言特性。

成员组成

胡格致 石曜铭 蔡合森

实验设计

设计思路

采用 **分层文法** 来表达不同优先级的运算（每层对应一种运算优先级，越深层优先级越高）。

在 **Lexer** 层对数值字面量（十六/十/八进制整数、小数与指数形式）进行严格划分并优先匹配浮点数以避免被拆分。

在解析阶段使用自定义 **ErrorListener + BailErrorStrategy**，遇到第一个致命错误立即报告并中止，保证“及时报错并返回非 0”。

各类型（int/float/double）分别维护表达式规则，便于在后续加入类型检查与代码生成。

实验实现

(1) 如何把表达式优先级体现在文法设计中？

采用**分层非终结符 + 左递归**实现优先级与结合性。：

- 整型表达式结构:

```
int_exp : int_exp_4 ;

int_exp_4
  : int_exp_4 Plus   int_exp_3   # int_4_3_plus
  | int_exp_4 Minus  int_exp_3   # int_4_3_minus
  |                               # int_4_3_none
  ;

int_exp_3
  : int_exp_3 Times  int_exp_2   # int_3_2_times
  | int_exp_3 Divide int_exp_2   # int_3_2_divide
  | int_exp_3 Remain int_exp_2   # int_3_2_remain
  |                               # int_3_2_none
  ;

int_exp_2
  : Plus   int_exp_2   # int_2_1_plus
  | Minus  int_exp_2   # int_2_1_minus
  |                               # int_2_1_none
  ;

int_exp_1
  : SBra int_exp_4 SKet    # int_1_4_braket
  | int_const                # int_1_int_const
  | func_call                # int_1_func_call
  | Ident                    # int_1_ident
  ;
```

int_exp_3 (* / %) 位于 int_exp_4 的子层，因此乘除在解析中先结合，表现为乘除优先于加减；

int_exp_4 的左递归形式 (int_exp_4 : int_exp_4 Plus int_exp_3 | ...) 保证同级运算的左结合性 (例如 $a - b - c$ 被解析为 $(a - b) - c$) ;

int_exp_2 作为一元层处理 +/-, int_exp_1 为 primary (括号、常量、函数调用、标识符) 。

将不同优先级放在不同层，解析时深层先合并、浅层后合并，从而自然体现优先级；左递归保证结合性。

(2) 如何设计数值常量的词法规则?

在 lexer 层定义了多个 token 与片段，负责匹配不同形式的数值常量，并在语法层通过 sign_* 等规则处理带符号情况。主要规则摘录并说明如下。

- 浮点 / 双精度相关 (grammar 中项) :

```

FractConst    : [0-9]* '.' [0-9]+ | [0-9]+ '.' ;
ExpPart       : [E|e] '+' [0-9]+ | [E|e] '-' [0-9]+ | [E|e] [0-9]+ ;

FloatConst    : FractConst ExpPart+ [F|f]
               | [0-9]+ ExpPart [F|f]
               ;
DoubleConst   : FractConst ExpPart+
               | [0-9]+ ExpPart
               ;
SignFloatConst : Plus FloatConst | Minus FloatConst | FloatConst ;
SignDoubleConst : Plus DoubleConst | Minus DoubleConst | DoubleConst ;

```

说明:

- FractConst 定义小数形式（例如 .5、1.、1.23）；ExpPart 定义指数部分（例如 E+10、e-3）；FloatConst/DoubleConst 组合这些片段表示带小数/带指数的浮点常量；
- 带符号的浮点通过语法层 SignFloatConst/SignDoubleConst 把 Plus/Minus token 与 FloatConst/DoubleConst 组合成带符号字面量（+3.14、-1.2e3）。
- 整数相关（grammar 中项）：

```

HexConst      : [0][x|X][0-9A-Fa-f]+ ;
DecConst      : [1-9] [0-9]* ;
OctConst      : [0] [0-7]* ;

int_const     : DecConst | OctConst | HexConst ;
sign_int_const : Plus int_const | Minus int_const | int_const ;

```

说明:

- HexConst 以 0x/0X 开头并跟十六进制数字；DecConst 表示十进制（非零开头）；OctConst 表示以 0 开头的八进制；
- int_const 在语法层合并三类整数 token；sign_int_const 在语法层把 +/- 与整数组合从而得到带符号整数字面量。
- 配合关系：
 - 表达式的 primary 层（如 int_exp_1/float_exp_1/double_exp_1）直接引用对应的字面量 token（int_const、FloatConst、DoubleConst），因此 lexer 所生成的 token 决定了语法如何识别字面量；
 - Plus/Minus 在 lexer 中是独立的 token（操作符/符号），在语法层通过 sign_* 与常量结合，这表明语法在处理一元符号时负责组合而非词法将符号并入常量。

结论：CACT.g4 用明确的 lexer token 将不同数值形式划分开，辅以语法层 sign_* 规则处理带符号，使数值字面量在词法与语法两层配合下被正确识别。

(3)如何实现异常处理以做到在发现语法错误时及时报错？（grammar 层面的可观察机制）

说明：CACT.g4 本身为 grammar 描述，不包含运行时代码（如异常抛出或退出）。但 grammar 将错误暴露的机制与位置明确化，使得运行时能够在首次出现错误时检测并报错。基于 CACT.g4，可观察到支持及时报错的点如下：

1. 词法阶段的暴露

- Lexer 明确列举合法 token（关键字、操作符、数字字面量、标识符、注释、空白等）。当输入字符序列无法匹配任何 lexer 规则时，词法器会处于“无法匹配”状态，该情况即为词法错误并会导致错误事件被上报（运行时层面会收到）。
- 例如不完整的数值字面量（无法匹配 FractConst/ExpPart/HexConst 等）会在词法阶段导致无法产生合法 token，从而触发错误。

2. 语法阶段的暴露

- Parser 的产生式对 token 序列有严格期待（如 `int_decl : INT int_def (Comma int_def)* Semi ;`；要求以 INT 开头并以分号结束）。当 token 流在某一位不在当前产生式允许的集合内时，解析器即识别为语法错误并产生错误事件。
- 由于大多数语句/声明具备明确结束符（`;` 或 `}`），常见错误（缺分号、参数不完整等）会在出现位置被迅速识别。

3. 语法结构有利于定位

- 入口规则 `comp_units` 与一条条明确的 `comp_unit/item` 规则使解析逐条进行，遇错时能精确定位到出错的语句与位置（行/列 由运行时 token 信息提供）。

结论：CACT.g4 在 grammar 层已通过明确的 lexer/ parser 规则把非法输入暴露出来——这些暴露点促成了运行时能在第一处词法或语法错误发生时及时检测并报告（timely reporting 的先决条件由 grammar 提供；具体的异常抛出/退出由运行时实现决定）。

总结

实验结果总结

- 表达式优先级：CACT.g4 通过分层 `*_exp_*` 非终结符与左递归明确体现优先级与左结合性；不同类型（int/float/double/bool）各自维护表达式层次，语法结构清晰。
- 数值字面量词法设计：CACT.g4 将字面量分为 HexConst/DecConst/OctConst（整数）与 FloatConst/DoubleConst（浮点），并用 FractConst/ExpPart 表示浮点内部结构；带符号常量由语法层 `sign_*` 组合。
- 及时报错能力（grammar 层）：grammar 本身通过严格的 lexer 与 parser 规则将非法输入暴露出来，满足在词法或语法首次出现错误时被运行时检测的先决条件。

